



When Replanning Becomes the Bottleneck: Budgeted Replanning for Embodied-Agents

Shuaijun Liu, Feiyang You, Xingwei Chen, Ningxin Su[†]

The Hong Kong University of Science and Technology (Guangzhou) | [†] ningxinsu@hkust-gz.edu.cn



1 Motivation

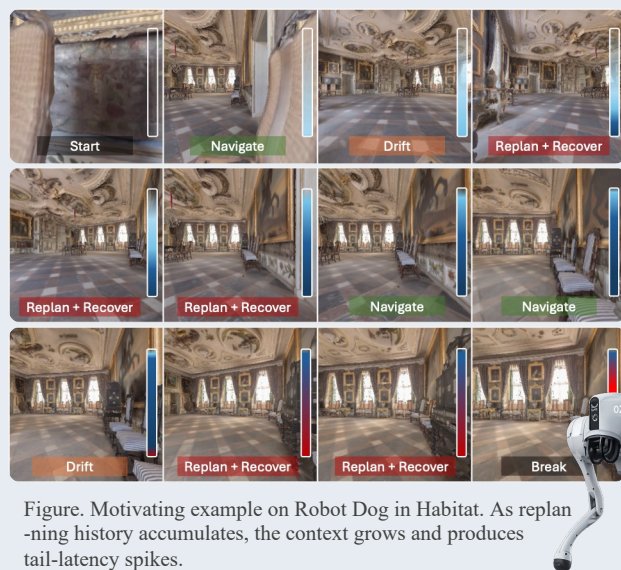


Figure. Motivating example on Robot Dog in Habitat. As replanning history accumulates, the context grows and produces tail-latency spikes.

Replanning is necessary for partial observability, drift, execution failures, and coordination hazards. But LLM/VLA replanning contexts grow with history, perception, memory, and agents.

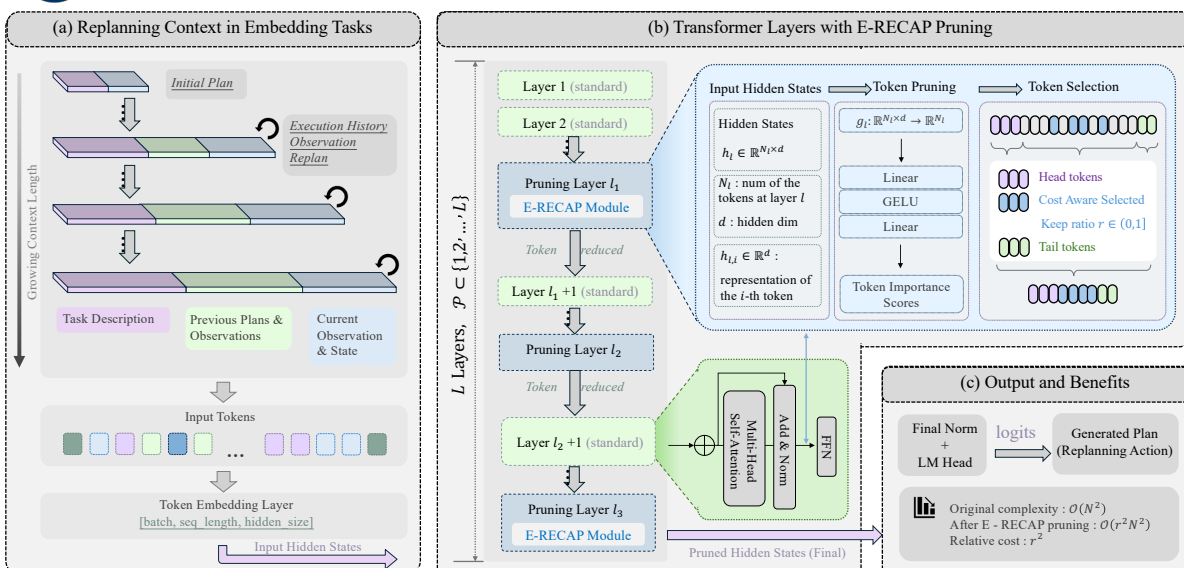
- ❖ Each call includes task specs, observations, failures, actions, and messages.
- ❖ Slow calls delay execution and trigger more replanning.
- ❖ Mean success hides latency tails, churn, deadline misses.
- ❖ BRACE evaluates call-level tokens, P95/P99 latency, and SLO violations.

BRACE = Budgeted Replanning for Agentic Control in Embodied Systems

2 Problem Formulation

- (1) Growing context. The replanning context grows as $N_t = N_0 + \sum_{i=1}^t \sum_{a=1}^K h_{i,a}$, requiring an explicit token budget B_t and latency target SLO_t .
- (2) Call-level metrics. Performance is measured by the SLO violation rate, $\text{mean}[\text{latency}_t > SLO_t]$, rather than episode averages.

4 E-RECAP Token Pruning



E-RECAP module. A lightweight predictor scores tokens from intermediate hidden states, with progressive pruning at selected layers.

3 BRACE Controller

BRACE maps trigger signals and short-horizon controller state to a replanning decision, mode, token budget, latency SLO. BRACE outputs budgets; the runner/pruner enforces them.

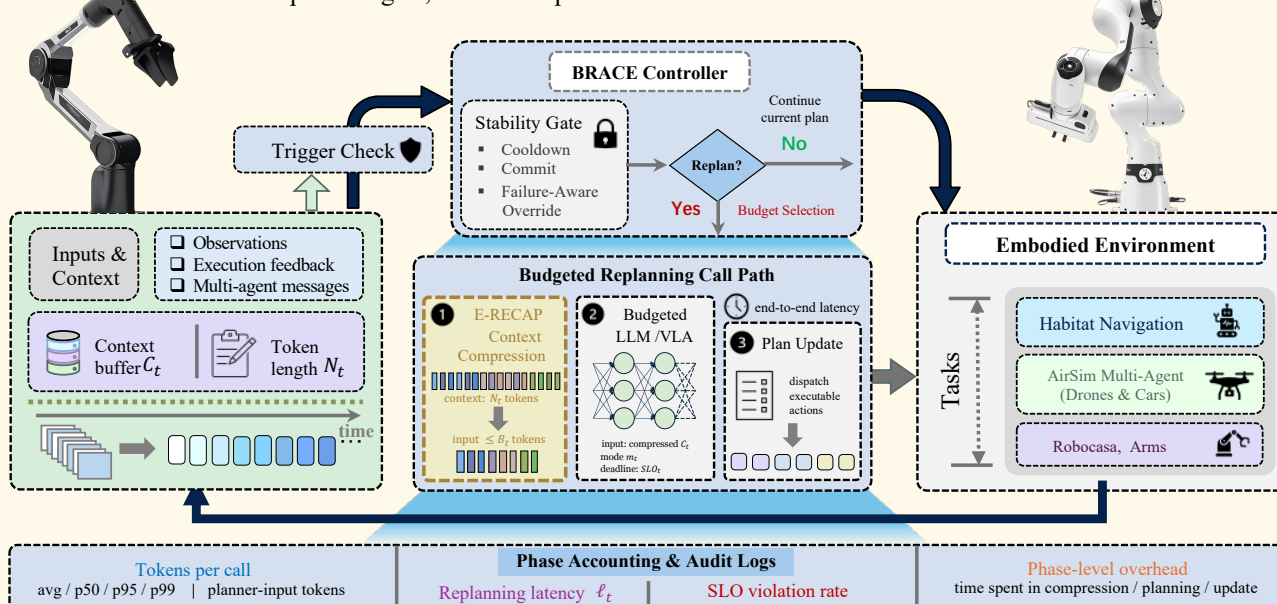
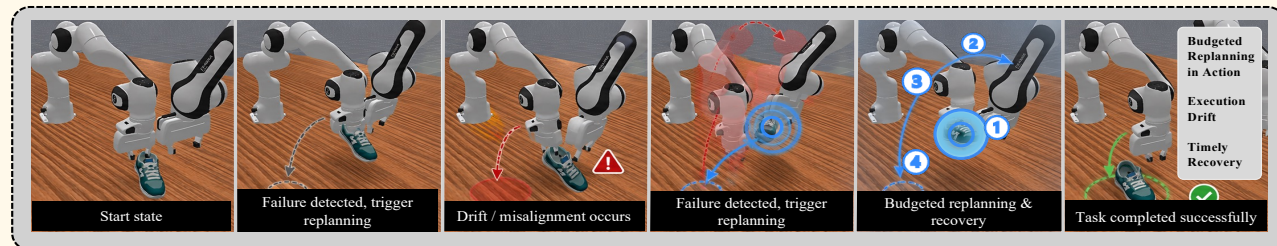


Figure. Overview of BRACE: a closed-loop controller that determines whether to invoke replanning and selects the per-call token budget and latency SLO, with E-RECAP as a composable token-pruning module on the replanning call path.

- Full** safety / deadlock recovery
- Partial** SLO-aware constrained context
- Reuse** commit to current subplan
- Defer** suppress churn under cooldown

Controller gates admit or suppress replanning, then compose optional modules such as E-RECAP, retrieval, caching, and communication compression.



Main Results (Ablations & Real Robot)

Method	Success	P95	SLO
Open-loop	0.0%	N/A	N/A
Frozen plan	0.0%	72.8	0.0%
No BRACE	0.0%	312.7	27.6%
BRACE + E-RECAP	80.0%	247.2	4.6%

Table. Harder RoboFactory: open-loop, frozen-plan, and No BRACE all fail; BRACE recovers by spending budget when recovery matters.

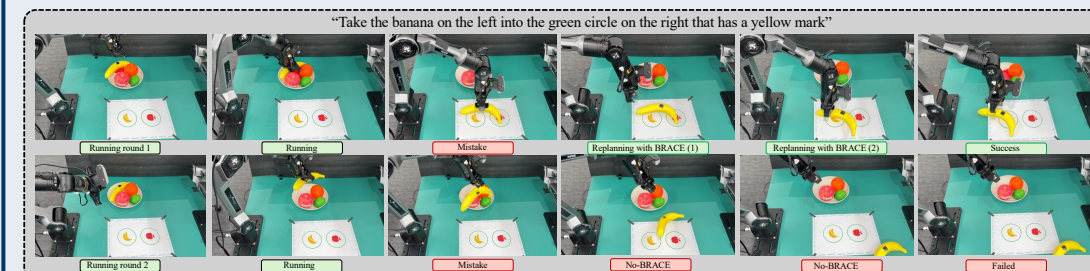
Task	Method	Succ.	SLO
PickFruit	No BRACE	24.0%	42.7%
PickFruit	BRACE	40.0%	18.6%
PushT	No BRACE	12.0%	61.3%
PushT	BRACE	32.0%	27.5%

Table. Physical execution: the same interface improves focused arm tasks and lowers SLO misses. Method Success P95 SLO Task Method

5 Main Results



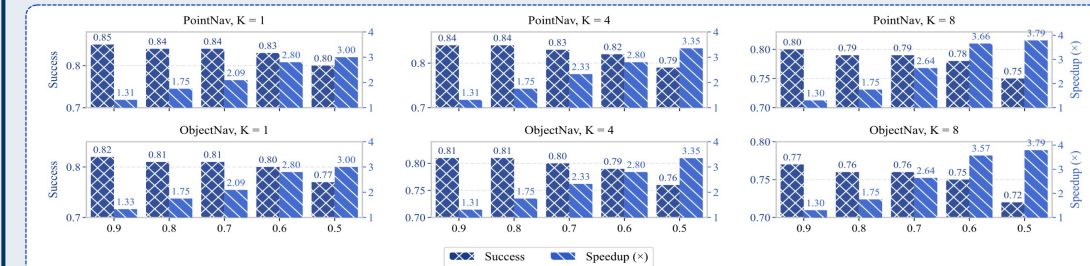
Figure. AirSim drones comparison. BRACE shortens replanning paths and reduces deadline misses.



Real-robot PICKFRUIT. BRACE replans twice after an execution failure and successfully recovers, while the baseline fails to recover.

Benchmark	Method	Tokens	P95	SLO viol.
Meta Habitat	No BRACE	235	2677	85.5%
Meta Habitat	BRACE + E-RECAP	20	2500	4.7%
RoboFactory	No BRACE	1566	1604	100.0%
RoboFactory	BRACE + E-RECAP	319	1213	50.0%
AirSim K=8	No BRACE	2934	8520	100.0%
AirSim K=8	BRACE + E-RECAP	1114	1640	4.7%

Success is saturated, but budgeted replanning sharply reduces deadline misses, highlighting the importance of reporting tail latency alongside success.



E-RECAP tradeoff between compression and performance under stronger pruning. The plot shows how latency and token savings change as the keep ratio decreases, alongside the resulting task-quality trend.

6 Conclusion

BRACE reframes replanning as a **budgeted control loop** with call-level accounting for **tokens, latency, and controller decisions**. A lightweight controller stabilizes replanning and selects **Bt / SLOt**, while **E-RECAP** progressively preserves head/tail context and high-utility tokens. **Per-call logs** provide auditability, and experiments on navigation, manipulation, coordination, and real robots demonstrate consistent gains.